```sql
/*
        ------------------------------
        -- Exploratory Data Analysis --
        ------------------------------
        01- Database Exploration
        02- Dimensions Explorations
        03- Date Exploration
        04- Measures Explorations
        05- Magnitude Analysis
        06- Ranking
        07- Trend Analysis (Change Over Time)
*/

USE DataWarehouseAnalytics;


/*
        ==============================
        == 01- Database Exploration ==
        ==============================
        - explore the structure of the database
        - just to have basic understanding about the database tables, views, columns,
*/

-- explore all objects in the database
SELECT * FROM INFORMATION_SCHEMA.TABLES;

-- explore all columns in the database
SELECT * FROM INFORMATION_SCHEMA.COLUMNS;



/*
        ================================
        == 02- Dimensions Explorations ==
        ================================
        - identifying the unique values or categories in each dimensions
        - recognizing how data might be grouped or segmented
        - which is useful for later analysis

        like
                DISTINCT Product
                DISTINCT Country
                DISTINCT Category
*/


-- let's explore the dimension values inside out database
-- take customers as example

-- explore all countries our customers come from
SELECT DISTINCT Country
FROM gold.dim_customers;


-- explore all product categories 'the major divisions'
SELECT DISTINCT
        category
FROM gold.dim_products;
```

```sql
-- see the category, sub-categories
SELECT DISTINCT
        category,
        subcategory
FROM gold.dim_products;


SELECT DISTINCT
        category,
        subcategory,
        product_name
FROM gold.dim_products;


/*
        ==========================
        == 03- Date Exploration ==
        ==========================
        - identify the earliest and latest dates (boundaries)
        - understand the scope of data and the timespan
        - MIN/MAX [date dimension]
*/


-- find the date of the first and last order
SELECT
        MIN(order_date) AS first_order_date,
        MAX(order_date) AS last_order_date,
        DATEDIFF(YEAR, MIN(order_date), MAX(order_date)) as order_range_years,
        DATEDIFF(MONTH, MIN(order_date), MAX(order_date)) as order_range_months,
        DATEDIFF(DAY, MIN(order_date), MAX(order_date)) as order_range_days
FROM gold.fact_sales;



-- find the youngest and oldest customer
SELECT
        MIN(birthdate) AS oldest_birthdate,
        DATEDIFF(YEAR, MIN(birthdate), GETDATE()) AS oldest_age,
        MAX(birthdate) AS youngest_birthdate,
        DATEDIFF(YEAR, MAX(birthdate), GETDATE()) AS youngest_age
FROM gold.dim_customers;


/*
        ==============================
        == 04- Measures Explorations ==
        ==============================
        - calculate and find out the key metric of the business (big numbers)
        - highest level of aggregation | lowest level of aggregation
*/


-- find the total sales
SELECT
        SUM(sales_amount) AS total_sales
FROM gold.fact_sales;
```

```sql
-- find how many items are sold
SELECT
        SUM(quantity) AS total_quantity
FROM gold.fact_sales;


--      find the average selling price
SELECT
        AVG(price) AS avg_price
FROM gold.fact_sales;


-- find the total number of orders
SELECT
        COUNT(order_number)
FROM gold.fact_sales;


-- find the total number of 'distinct orders'
SELECT
        COUNT(DISTINCT order_number)
FROM gold.fact_sales;


-- find the total number of products
SELECT
        COUNT(product_key) AS total_products
FROM gold.dim_products;


-- find the total number of customers
SELECT
        COUNT(customer_key) AS total_customers
FROM gold.dim_customers;


-- find the total number of customers that has placed an order
SELECT
        COUNT(DISTINCT customer_key) AS total_customers
FROM gold.fact_sales;


/*
        ===============================================================
        == Generate Report That Shows All Key Metrics Of The Business ==
        ===============================================================
*/
SELECT 'total sales' AS measure_name, SUM(sales_amount) AS measure_value FROM
gold.fact_sales
UNION ALL
SELECT 'total quantity', SUM(quantity) FROM gold.fact_sales
UNION ALL
SELECT 'average price', AVG(price) FROM gold.fact_sales
UNION ALL
SELECT 'number of orders', COUNT(DISTINCT order_number) FROM gold.fact_sales
UNION ALL
SELECT 'number of products', COUNT(product_name) FROM gold.dim_products
UNION ALL
SELECT 'number of customers', COUNT(customer_key) FROM gold.dim_customers;
```

```sql
/*
        ============================
        == 05- Magnitude Analysis ==
        ============================
        - compare the measure values by categories, it help us to understand the
importance of differnt categories
        - such as
                total sales by country
                total quantity by category
                average price by product
                total orders by customer
*/

-- find total customers by countries
SELECT
        country,
        COUNT(customer_key) AS total_customers
FROM gold.dim_customers
GROUP BY country
ORDER BY total_customers DESC;


-- find total customers by gender
SELECT
        gender,
        COUNT(customer_key) AS total_customers
FROM gold.dim_customers
GROUP BY gender
ORDER BY total_customers DESC;


-- find total products by categories
SELECT
        category,
        COUNT(product_key) AS total_products
FROM gold.dim_products
GROUP BY category
ORDER BY total_products DESC;



-- what is the average costs in each category?
SELECT
        category,
        AVG(cost) AS avg_costs
FROM gold.dim_products
GROUP BY category
ORDER BY avg_costs DESC;


-- what is total revenue generated for each category?
-- usually starts with the fact table, then LEFT JOIN the dimensions to it
-- NOTE: LEFT JOIN kept intentionally - preserves any fact rows without a matching
-- product/customer record so we don't silently lose unmatched sales from totals
SELECT
        p.category,
        SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_products AS p
ON p.product_key = f.product_key
GROUP BY p.category
```

```sql
ORDER BY total_revenue DESC;



-- find the total revenue generated by each customer
SELECT
        c.customer_key,
        c.first_name,
        c.last_name,
        SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_customers AS c
ON c.customer_key = f.customer_key
GROUP BY
        c.customer_key,
        c.first_name,
        c.last_name
ORDER BY total_revenue DESC;


-- what is the distribution of sold items across countries
SELECT
        c.country,
        SUM(f.quantity) AS total_sold_items
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_customers AS c
ON c.customer_key = f.customer_key
GROUP BY
        c.country
ORDER BY total_sold_items DESC;



/*
        ============================
        == 06- Ranking Analysis ==
        ============================
        - order the values of dimensions by measure
        - identify the top N performers | bottom N perform
                rank countries by total sales
                top 5 products by quantity
                bottom 3 customers by total orders
*/


-- which 5 products generate the highest revenue
SELECT TOP 5
        p.product_name,
        SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_products AS p
ON p.product_key= f.product_key
GROUP BY
        p.product_name
ORDER BY total_revenue DESC;
```

```sql
-- what are the 5 worst-performing products in terms of sales
SELECT TOP 5
        p.product_name,
        SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_products AS p
ON p.product_key= f.product_key
GROUP BY
        p.product_name
ORDER BY total_revenue ASC;



-- which 5 subcategory generate the highest revenue
SELECT TOP 5
        p.subcategory,
        SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_products AS p
ON p.product_key= f.product_key
GROUP BY
        p.subcategory
ORDER BY total_revenue DESC;


-- which 5 products generate the highest revenue using window functions
SELECT *
FROM (
        SELECT
                p.product_name,
                SUM(f.sales_amount) AS total_revenue,
                ROW_NUMBER() OVER(ORDER BY SUM(f.sales_amount) DESC) AS rank_products
        FROM gold.fact_sales AS f
        LEFT JOIN gold.dim_products AS p
        ON p.product_key= f.product_key
        GROUP BY
                p.product_name
) AS t
WHERE rank_products <= 5;



-- find the top 10 customers who have generated the highest revenue
SELECT TOP 10
        c.customer_key,
        c.first_name,
        c.last_name,
        SUM(f.sales_amount) AS total_revenue
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_customers AS c
ON c.customer_key = f.customer_key
GROUP BY
        c.customer_key,
        c.first_name,
        c.last_name
ORDER BY total_revenue DESC;
```

```sql
-- find 3 customers with the fewest orders placed
-- FIX: was ORDER BY total_orders DESC, which returned the customers with the
-- MOST orders instead of the fewest. Corrected to ASC.
SELECT TOP 3
        c.customer_key,
        c.first_name,
        c.last_name,
        COUNT(DISTINCT order_number) AS total_orders
FROM gold.fact_sales AS f
LEFT JOIN gold.dim_customers AS c
ON c.customer_key = f.customer_key
GROUP BY
        c.customer_key,
        c.first_name,
        c.last_name
ORDER BY total_orders ASC;


/*
        ===============================================
        == 07- Trend Analysis (Change Over Time) ==
        ===============================================
        - track how a measure evolves over time
        - useful for spotting seasonality, growth, and decline
        - combined with window functions for running totals and month-over-month
comparisons
*/

-- total sales by year and month
SELECT
        YEAR(order_date) AS order_year,
        MONTH(order_date) AS order_month,
        SUM(sales_amount) AS total_sales,
        COUNT(DISTINCT customer_key) AS total_customers,
        SUM(quantity) AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY order_year, order_month;


-- same trend using DATETRUNC for a cleaner single date column (SQL Server 2022+)
SELECT
        DATETRUNC(MONTH, order_date) AS order_month,
        SUM(sales_amount) AS total_sales,
        COUNT(DISTINCT customer_key) AS total_customers,
        SUM(quantity) AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY DATETRUNC(MONTH, order_date)
ORDER BY order_month;
```

```sql
-- running total of sales over time, plus month-over-month comparison using LAG()
WITH monthly_sales AS (
        SELECT
                DATETRUNC(MONTH, order_date) AS order_month,
                SUM(sales_amount) AS total_sales
        FROM gold.fact_sales
        WHERE order_date IS NOT NULL
        GROUP BY DATETRUNC(MONTH, order_date)
)
SELECT
        order_month,
        total_sales,
        SUM(total_sales) OVER (ORDER BY order_month) AS running_total_sales,
        LAG(total_sales) OVER (ORDER BY order_month) AS prev_month_sales,
        total_sales - LAG(total_sales) OVER (ORDER BY order_month) AS
month_over_month_change
FROM monthly_sales
ORDER BY order_month;
```